

Data Analysis in Physical Oceanography

LECTURE 2

SuSe26

Eleanor Frajka-Williams, Office 244 (Bu53)

Eleanor.Frajka@uni-hamburg.de

Data analysis techniques for observational data

Content: Equip you to approach oceanography from an experimental standpoint.

Intended participants: oceanography students

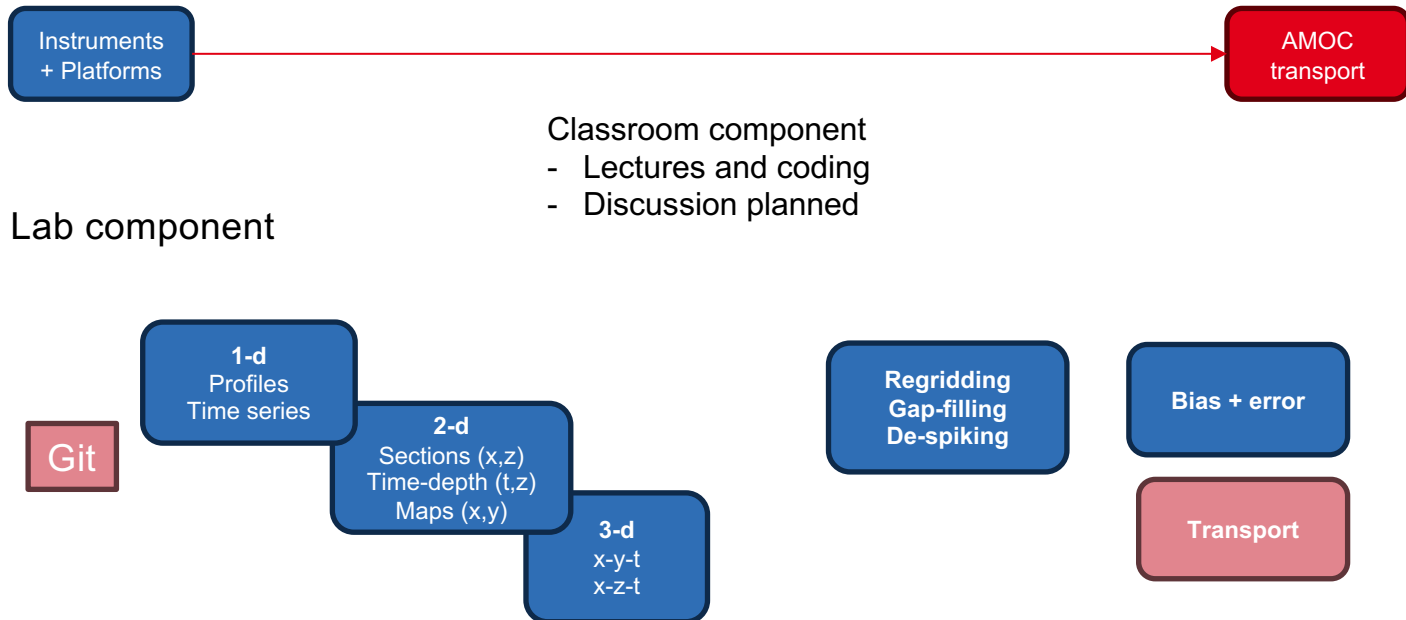
- who will go to sea in the near future
- who will be modellers or theoreticians
- who will become seagoing scientists

Case studies (2026 assignments): Observing the ocean circulation using moored observations

Practical skills:

- Scientific programming using **python** – for robust and reuseable code
 - Code structure, modules, functions + **tests**
- **Git** for version control and sharing code
 - Reviewing code, making and approving pull requests, handling issues
- **AI tools** allowed: Claude code, Copilot, ChatGPT, etc. Use them wisely. See “tests” above.

How will we get there? Course plan...



Assessments + expected deadlines

Activity	Percentage
Assignment 1: Analysis of AMOC	30%
Assignment 2: Calculating transport from moored arrays	60%
Exercise / participation	10%
Total	100%

- Assignment 1: Due June 19
- Assignment 2: Due Sept 4

Assignments will be submitted as git repositories.

1. Working tests
2. Working demo notebook
3. Modules for functions
4. Short report (figures + text) as report.md

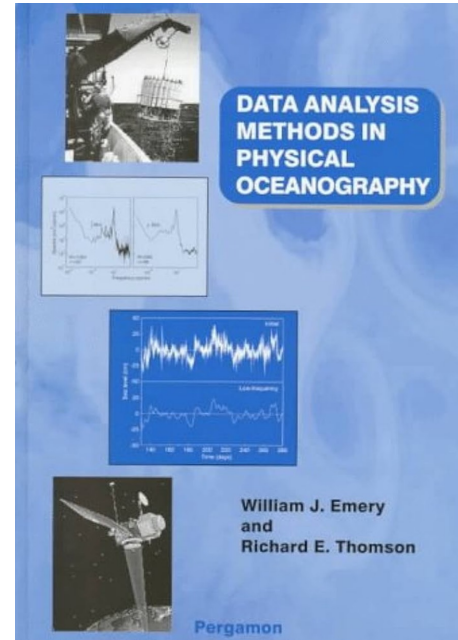
Exercises/Participation will include

- 1 figure discussion session (plan to prepare a figure + give feedback to peers) – deadline June 8
- Peer code feedback – deadline July 3

Textbook

- Emery + Thomson, Data analysis for physical oceanographers (2001, 2014, ..)
- Available at UHH
- Log in with UHH at <https://www.sciencedirect.com/book/9780444507563/data-analysis-methods-in-physical-oceanography>

Multiple chapters!



Course meeting times (Irregular – see Stine)

Mostly Mondays:

- 08:30 – 12:00 in GEOM 433

Preliminary sessions in April

- 13 April: Overview of tools
- **Before** April 27 -
 - Get VSCode, python, environment management working on your computer
 - Add your GitHub username to this sheet:
<http://bit.ly/4v9yPWe>
(create an account if you don't have one)
- 27 April: Troubleshooting

1	Mon, 13. Apr. 2026	10:15	11:45	Geom 433
2	Mon, 27. Apr. 2026	10:15	11:45	Geom 433
3	Mon, 18. May 2026	08:30	12:00	Geom 433
4	Mon, 1. Jun. 2026	08:30	12:00	Geom 433
5	Mon, 8. Jun. 2026	08:30	12:00	Geom 433
6	Mon, 6. Jul. 2026	08:30	12:00	Geom 433
7	Mon, 31. Aug. 2026	08:30	12:00	Geom 433

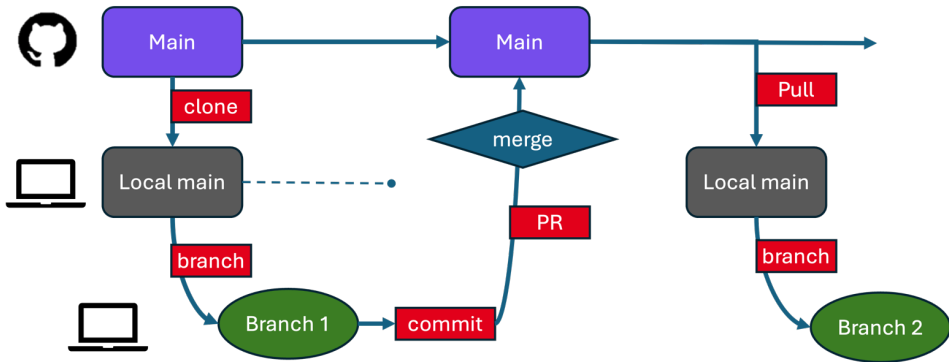
By 18 May, everyone should have all tools installed.

For your assignment, recommended:

SOLO GIT WORKFLOW

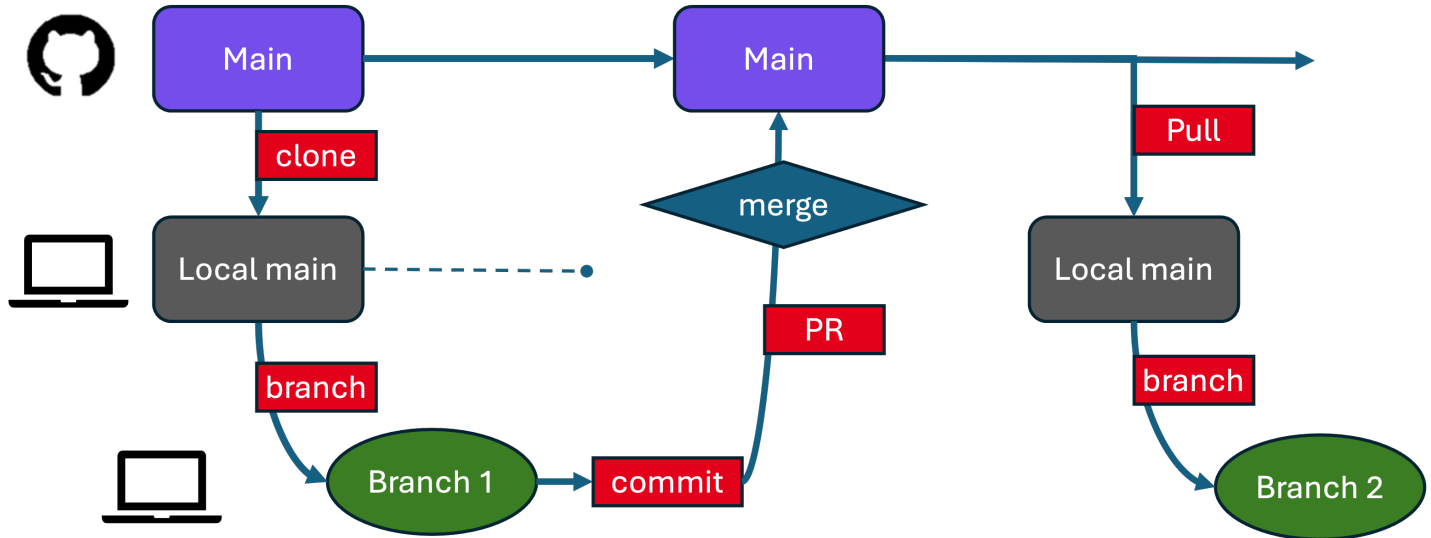
Solo Git workflow

- This guide shows how to work on your own project with Git. You'll learn how to:
 - create branches so you can edit code without breaking what works
 - sync your project across devices (laptop, desktop, server)
 - avoid merge conflicts and lost commits
- It assumes the repo is yours (not a fork) and that you're working alone or in a small team.



Workflow

A typical GitHub branch and pull request workflow (for solo working). You work in a branch of **your main**; the important copy of the code (and its history) is in the main branch.



1. Keep **main** clean

- The main branch should always reflect your most stable version — the code you'd be happy for someone else to run.
- Start each new task or fix on a **new branch** off main. That way, your working version stays safe while you experiment.
- **In VS Code:**
 - open the Source Control panel
 - click the branch name in the bottom-left status bar
 - select main, then click + *Create new branch* and give it a name
- **Or in the terminal:**

```
bash
```

```
git checkout main          # switch to main
git pull                   # get the latest from GitHub
git checkout -b feature-name # create and switch to a new branch
```

- Pick a branch name that says what you're doing: add-plotting-utils, update-docs, fix-logging-bug. Or just number them: eleanor-patch-1, eleanor-patch-2.

2. Work on a branch

- Make your changes on the new branch — editing code, fixing bugs, writing docs, whatever the task is. Then save and commit.
- **In VS Code:**
 - save your files (⌘S / Ctrl+S)
 - open the Source Control panel
 - type a short commit message
 - click  **Commit**
- **Or in the terminal:**

```
bash
```

```
git add . # stage all modified files  
git commit -m "feat: add new plotting utils" # save them with a message
```

- Forgot to save a file before committing? No problem — just save it and make another commit. Small, frequent commits are better than one big one anyway.

3. Option A: Merge the branch directly

- Once you're happy with the work on your feature branch, you can merge it straight into main from your own machine, then push the result to GitHub.
- Quick and simple — best when you don't need a review step.
- **In VS Code:**
 - switch to main (branch menu in the bottom-left status bar)
 - in the Source Control panel, click the ... menu → **Branch** → **Merge...**
 - select your feature branch
 - click **Sync Changes** to push to GitHub
- **Or in the terminal:**

```
bash

git checkout main      # switch to main
git pull               # make sure it's up to date
git merge feature-name # merge your branch in
git push               # send the result to GitHub
```

3. Option B: Open a pull request (recommended, even solo)

- Instead of merging on your own machine, push your branch to GitHub and open a pull request to merge it into main.
- Why bother when you're working alone?
 - you can preview the changes as a clean diff before they land
 - any CI (continuous integration) tests run automatically, catching problems before they reach main
 - the PR becomes a permanent record of what changed and why
- If you're using this template, CI is already set up: the workflow lives in *.github/workflows/tests.yml*, and the tests it runs are in *tests/*. Every push to a PR re-runs them.

Step 1 — push your branch to GitHub.

- In VS Code:
 - save and commit your changes
 - in the Source Control panel, click **Publish Branch** (first push) or **Sync Changes** (subsequent pushes)
 - if asked to pick a remote, choose origin

3. Option B: Open a pull request (recommended, even solo)

- Or in the terminal:

```
bash
```

```
git push -u origin feature-name # push and set the upstream
```

- The **-u** is only needed the first time you push a new branch; after that, plain **git push** works.

Step 2 — open the PR on GitHub.

- go to your repo: https://github.com/YOUR_USERNAME/YOUR_REPO
- click **Compare & pull request** in the banner that appears
- write a short description of what you changed and why
- click **Create pull request**

Step 3 — wait for tests, then merge.

- check the status of the automated tests at the bottom of the PR
- if they pass, click **Merge pull request**
- if they fail (or you spot something you want to change), commit and push again on the same branch — the PR updates automatically

4. Rinse and repeat

- For every new piece of work:
 - create a new branch off main
 - commit as you go
 - merge back in (directly or via PR)
 - push regularly
- The point is to keep changes small and isolated. That makes them easier to review, easier to undo, and less likely to clash with other work.

(Optional) Sync across devices

If you work on more than one machine — say a laptop in the field and a desktop at the office — Git is how you keep them in sync. The rule is simple: **push before you stop, pull before you start.**

On device A (when you finish working):

- VS Code: click **Sync Changes** in the Source Control panel.
- Terminal:

```
bash
git push origin main
```

On device B (when you sit down to work):

- VS Code: click the ... menu in Source Control → **Pull**.
- Terminal:

```
bash
git pull origin main
```

To sync a feature branch (not just **main**):

```
bash

# on device A
git push origin feature-name

# on device B
git checkout feature-name
git pull origin feature-name
```


Cheatsheet: handy Git commands (or ask copilot / Claude)

bash

`git status` # what's the current state of my working directory?

`git branch` # list all local branches

`git branch -d name` # delete a branch (only if already merged)

`git branch -D name` # force-delete a branch (even if not merged) – careful

`git log --oneline` # compact history of recent commits

`git log --oneline -10` # just the last 10

`git diff` # show changes not yet staged

`git diff --staged` # show changes staged for the next commit

`git restore filename` # discard unsaved changes to a file – careful, can't undo

`git restore --staged filename` # unstage a file (keeps your edits)

SPECTRA, FILTERING

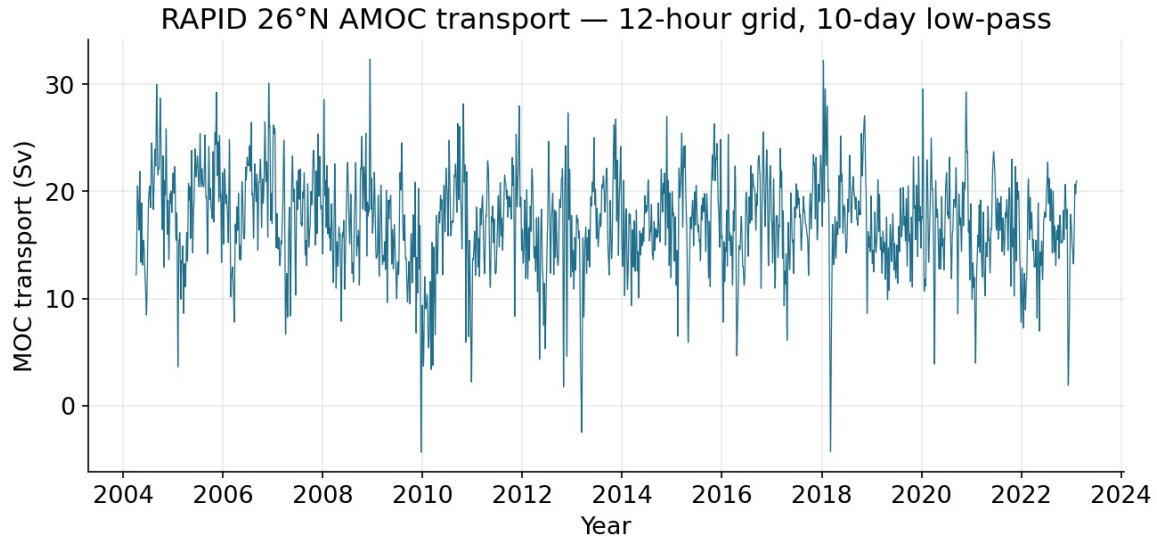
This lecture: spectra & filtering

- The grid sets what could be resolved; the filter sets what is actually there
- Estimate the spectrum honestly: periodogram → tapering → Welch → confidence intervals → Parseval
- Design filters that don't lie: Butterworth, Lanczos, Loess vs the boxcar
- Running example throughout: RAPID 26° N AMOC transport

Why work in the frequency domain?

- Time-domain statistics (mean, variance) say how much a series varies — not on what timescales
- Variance measures the total amount of variability (the square of the standard deviation)
- The power spectrum shows how that variance is distributed across frequency
- A frequency band is a range of frequencies; the spectrum gives the variance in each band
- So we can separate wind/Ekman, eddy, seasonal and interannual signals by their timescale

Our series: 26° N AMOC transport



12-hour grid, 10-day low-pass, 2004–2023; mean ≈ 17 Sv

Code — load the data

```
from amocatlas import read
import numpy as np

ds = read.rapid()          # download & cache RAPID 26N
print(ds)                  # explore variables, units, grid

moc = ds["moc_mar_hc10"]   # overturning transport (Sv)
x = moc.interpolate_na("time").values # fill gaps
fs = 2.0                   # samples/day (12-hour grid)
```

What the grid could resolve

1 · SAMPLING AND THE NYQUIST FREQUENCY

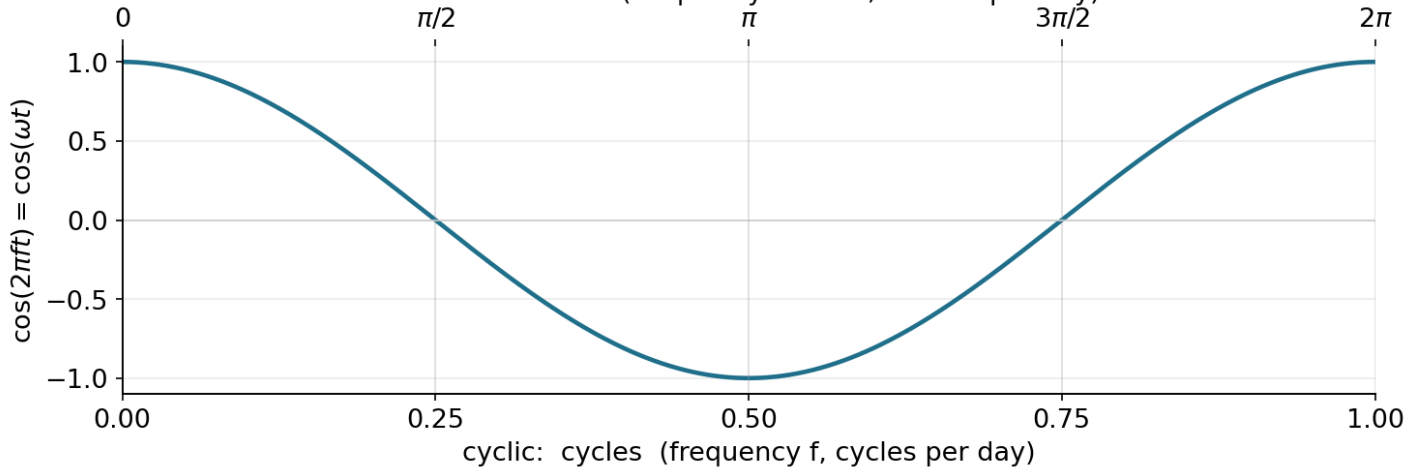
The sampling theorem

- A sampling interval Δt sets the Nyquist frequency $f_N = 1 / (2\Delta t)$: the highest frequency a grid can represent
- Frequencies are counted in cycles per day (cpd): 1 cpd is one full oscillation per day
- For $\Delta t = 12$ hours, $f_N = 1$ cpd — the fastest resolvable period is one day
- The Discrete Fourier Transform (DFT) — the finite-sample Fourier transform — represents frequencies from 0 up to f_N , and nothing higher
- Variability faster than f_N cannot be resolved; sampled too coarsely it aliases (next slide)

Frequency: cycles vs radians

One oscillation: 1 cycle = 2π radians (period $P = 1/f = 2\pi/\omega$)

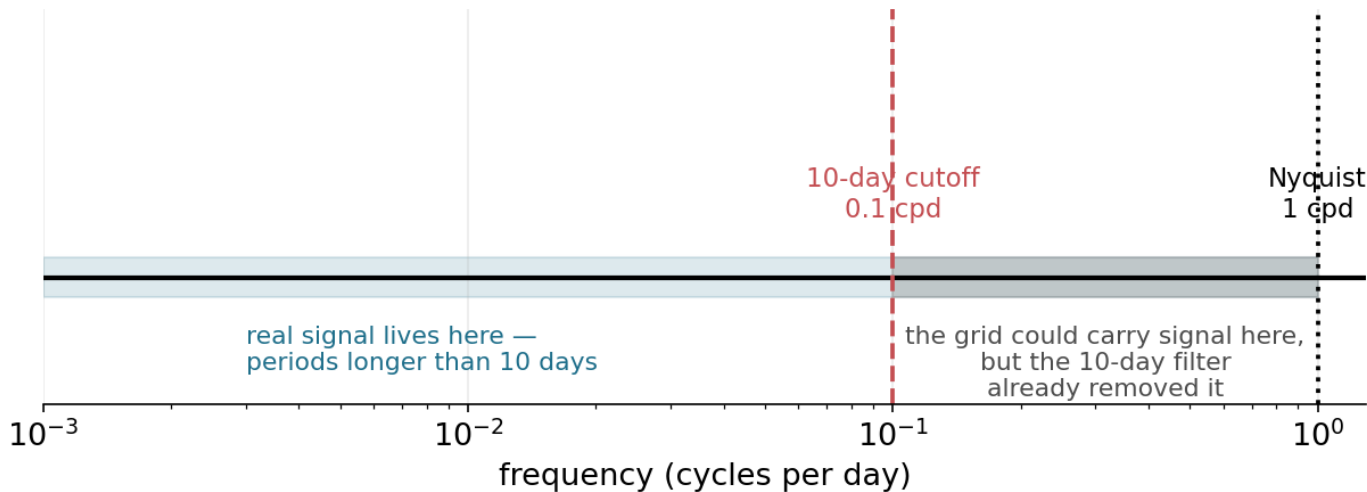
radian: radians (frequency $\omega = 2\pi f$, radians per day)



Cyclic frequency f (cycles/time, e.g. cpd) and radian frequency $\omega = 2\pi f$ (rad/time) describe the same oscillation

12-hour grid $\rightarrow$ Nyquist = 1 cpd

What a 12-hour interval could resolve vs. what the filter removed

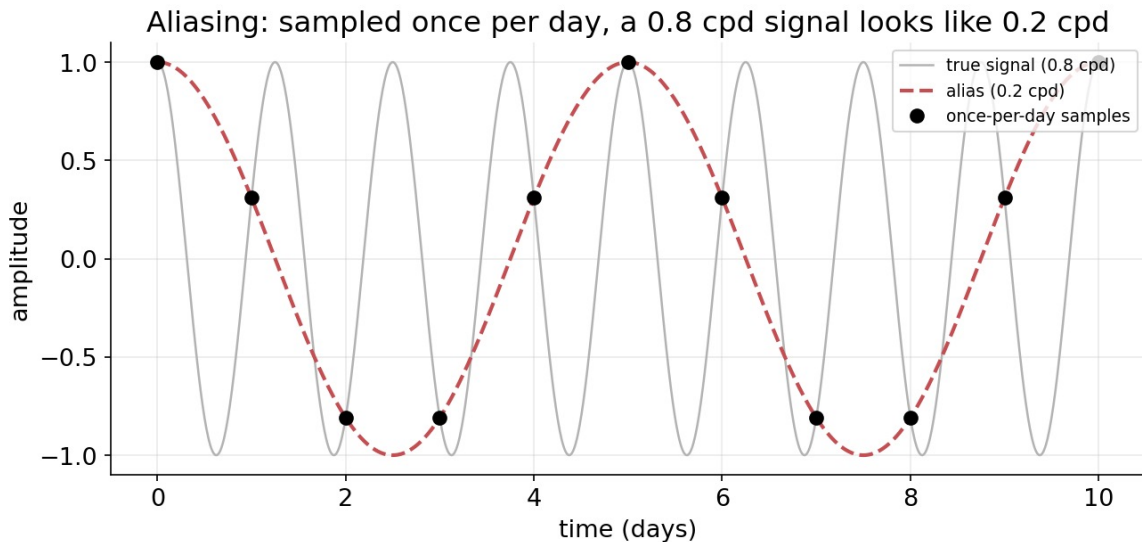


The grid defines the resolvable band — nothing more

Aliasing and a common misconception

- Energy above f_N folds back to lower frequencies (aliasing)
- Filtering before sampling prevents it — exactly what was done here
- Misconception: “12-hour data resolves everything down to 12 hours”

Aliasing: a fast signal in disguise



Sampled once per day, a 0.8 cpd signal is indistinguishable from 0.2 cpd

What is actually there

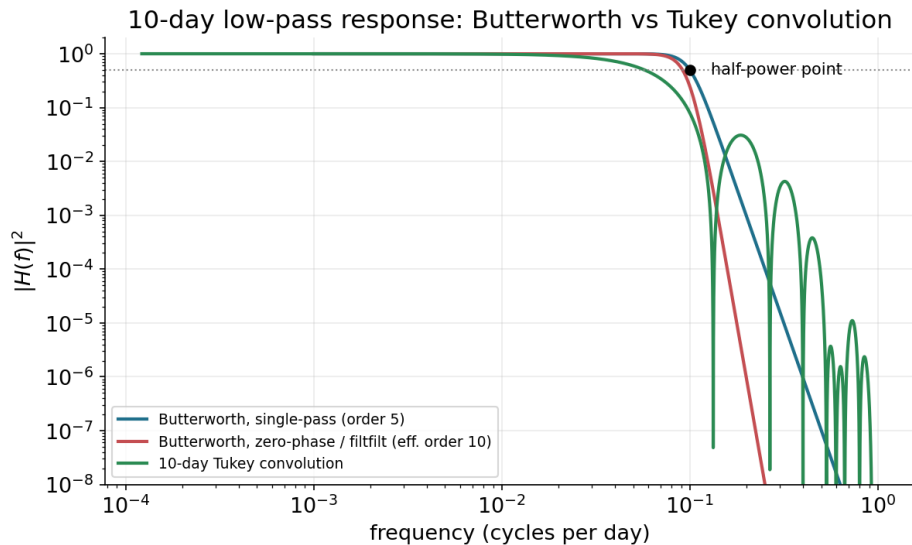
2 · THE FILTER SETS THE INFORMATION

These data are 10-day low-pass filtered

- RAPID low-pass filtered these data (10-day cutoff) before you ever see them
- $S_{in}(f)$: spectrum of the true signal (before filtering); $S_{out}(f)$: spectrum of the data (after filtering)
- Filtering scales the spectrum frequency-by-frequency by the filter's power response $|H(f)|^2$:
- Where $|H|^2 \approx 1$ the signal passes; where $|H|^2 \approx 0$ (above ~ 0.1 cpd) it is removed

$$S_{out}(f) = |H(f)|^2 S_{in}(f)$$

Butterworth response: a clean rolloff



Butterworth (maximally flat passband) vs a 10-day Tukey convolution

Code — Butterworth response

```
import numpy as np, matplotlib.pyplot as plt

f  = np.logspace(-3, 0, 600)    # cycles/day
n  = 5                          # filter order
H2 = 1 / (1 + (f/0.1)**(2*n))   # |H|^2, 0.1 cpd
H2_filtfilt = H2**2            # zero-phase: squared

plt.loglog(f, H2_filtfilt)
```


From the naive estimate to a trustworthy one

3 · ESTIMATING THE SPECTRUM HONESTLY

The periodogram

- The periodogram is the squared Discrete Fourier Transform, at $N/2$ frequencies
- Each value uses two coefficients (one sine, one cosine) $\rightarrow$ 2 degrees of freedom
- Standard deviation = mean (a chi-squared estimate, $\nu = 2$) $\rightarrow$ very spiky; we must average

$$\hat{S}(f_k) = \frac{\Delta t}{N} \left| \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} \right|^2$$

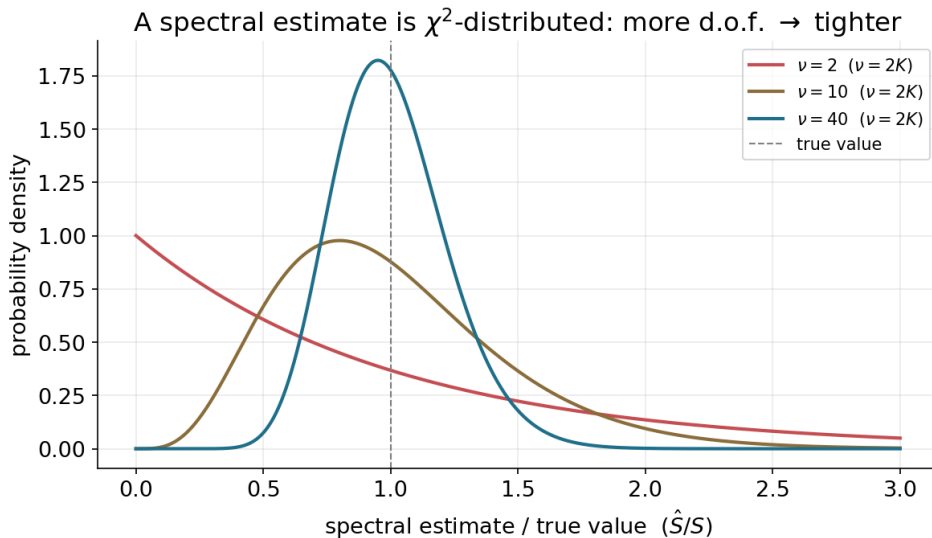
$\hat{S}$: estimate of the power spectrum (the hat $\wedge$ marks an estimate from data)

x_n : the data samples, $n = 0, 1, \dots, N-1$; N : number of samples

Δt : the sampling interval (time between samples)

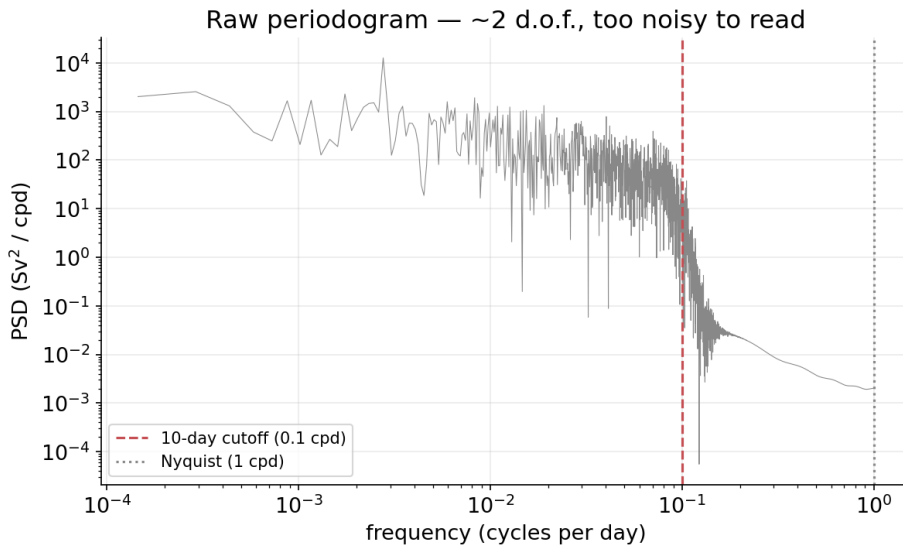
$f_k = k/(N\Delta t)$: the discrete frequencies, $k = 0, 1, \dots, N/2$

What is a chi-squared estimate?



Each spectral estimate scatters around the true value; more degrees of freedom $\rightarrow$ tighter

Periodogram: too noisy to read



The variance hides the rolloff knee and the floor

Code — periodogram

```
from scipy import signal

fs = 2.0      # sampling rate, samples/day (12-h grid)
f, Pxx = signal.periodogram(
    x, fs=fs, detrend="linear")  # window="boxcar"

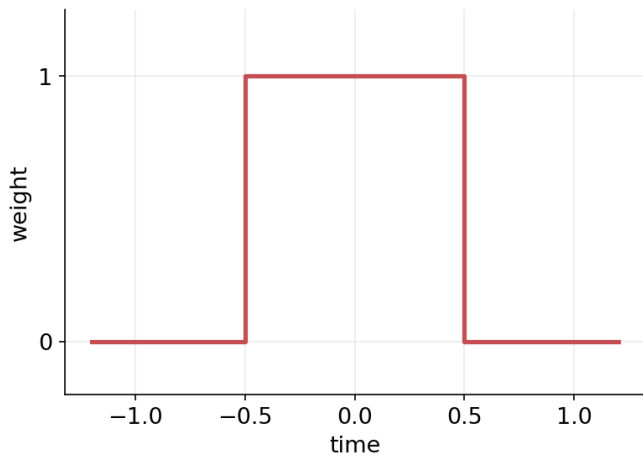
plt.loglog(f, Pxx)                # ~2 d.o.f., noisy
```

Leakage: the finite-record problem

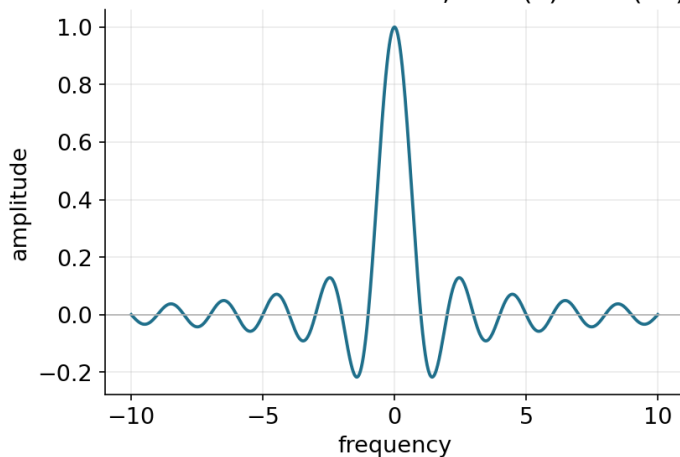
- A finite record is the true series times a boxcar window
- In frequency space that convolves each spectral line with a sinc
- $\text{sinc}(x) = \sin(x)/x$ — a tall central peak with slowly decaying ripples (sidelobes)
- Those sidelobes smear power into distant bins — that is leakage

A finite record and its transform

A finite record = a boxcar window

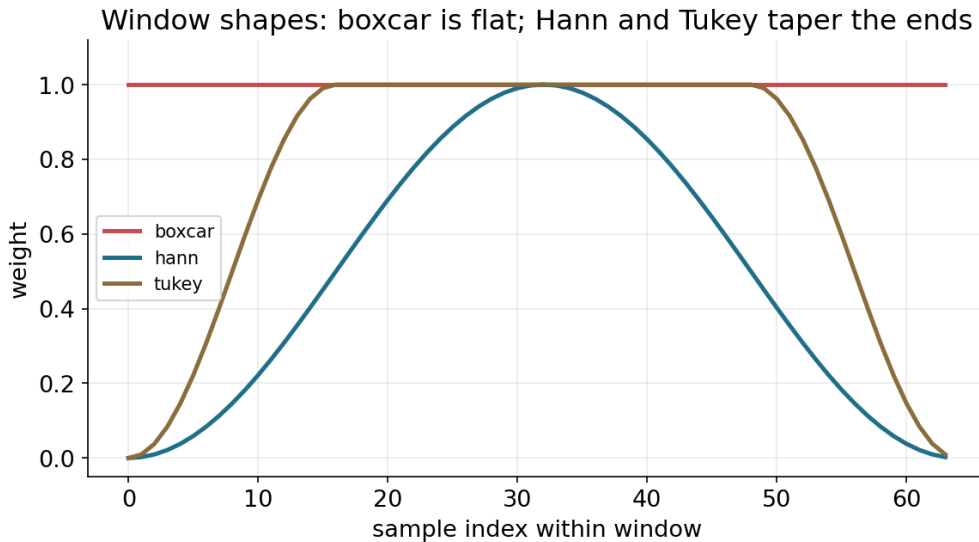


Its Fourier transform = a sinc, $\text{sinc}(x) = \sin(\pi x)/(\pi x)$



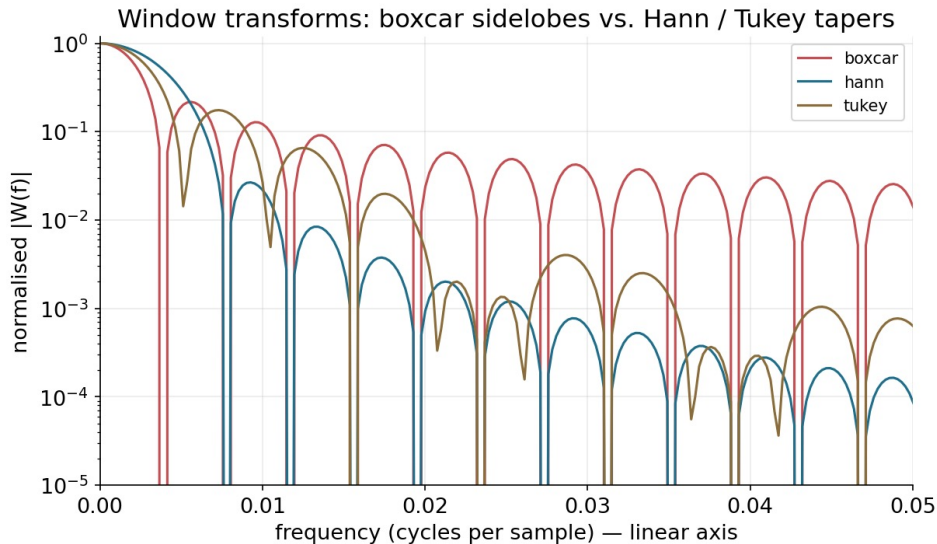
Cutting the record to a boxcar convolves the spectrum with a sinc — its sidelobes are leakage

What the windows look like



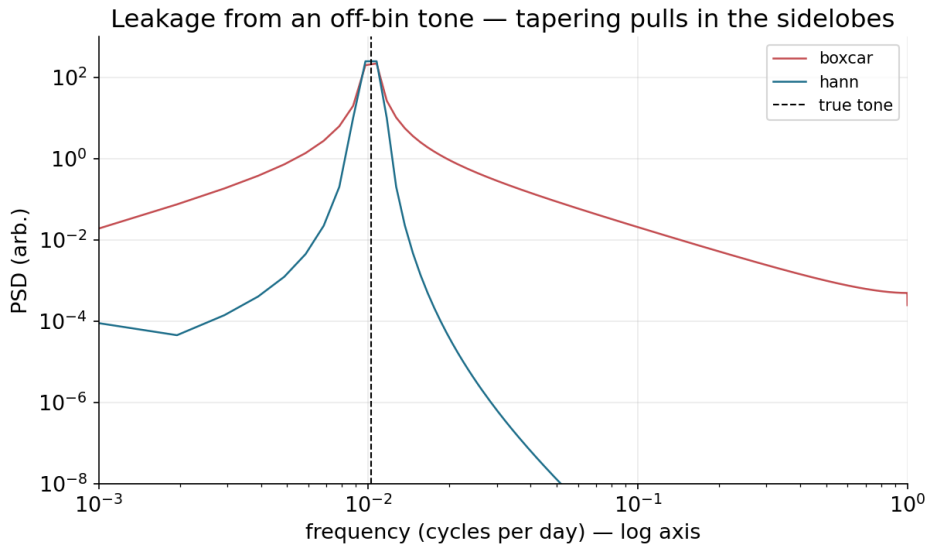
boxcar is flat; Hann and Tukey taper to zero at the ends

Window transforms: sidelobes vs taper



Linear frequency axis (log can't show the $f=0$ main lobe): boxcar leaks, Hann/Tukey don't

Leakage in action: a synthetic tone



Now on log-log axes: the Hann taper drops the leakage floor far below the boxcar

Code — leakage & tapering

```
n, dt = 2048, 0.5
t = np.arange(n) * dt
tone = np.cos(2*np.pi*0.06*t)    # off-bin tone

for win in ("boxcar", "hann"):
    f, P = signal.periodogram(
        tone, fs=fs, window=win)
    plt.semilogy(f, P, label=win)
```

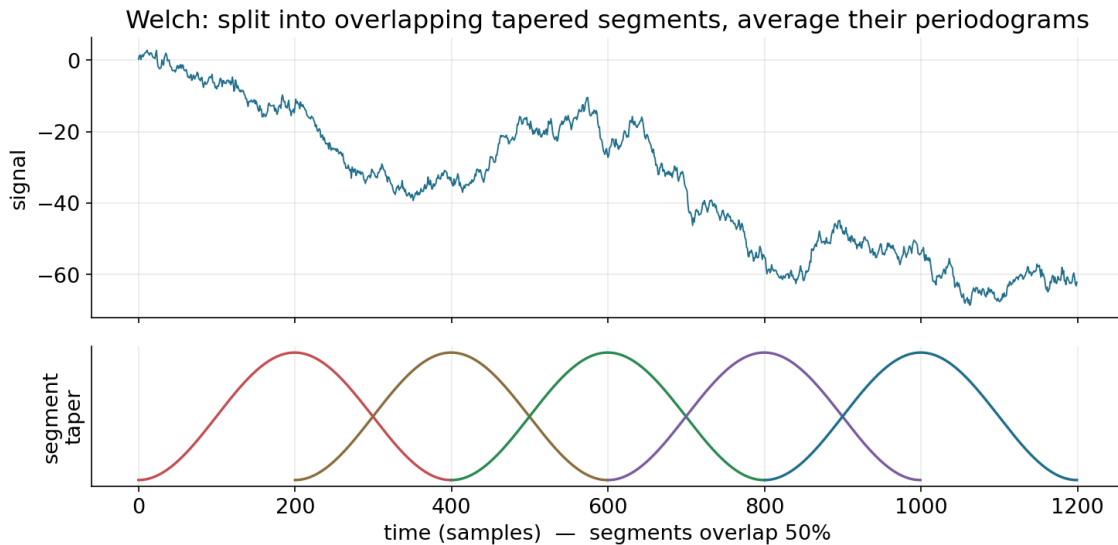
Tapering: the trade-off

- Wider main lobe (a little less resolution)
- Much lower sidelobes (much less leakage)
- Total power is conserved — leakage only redistributes it (Parseval)

Welch: overlapped segment averaging

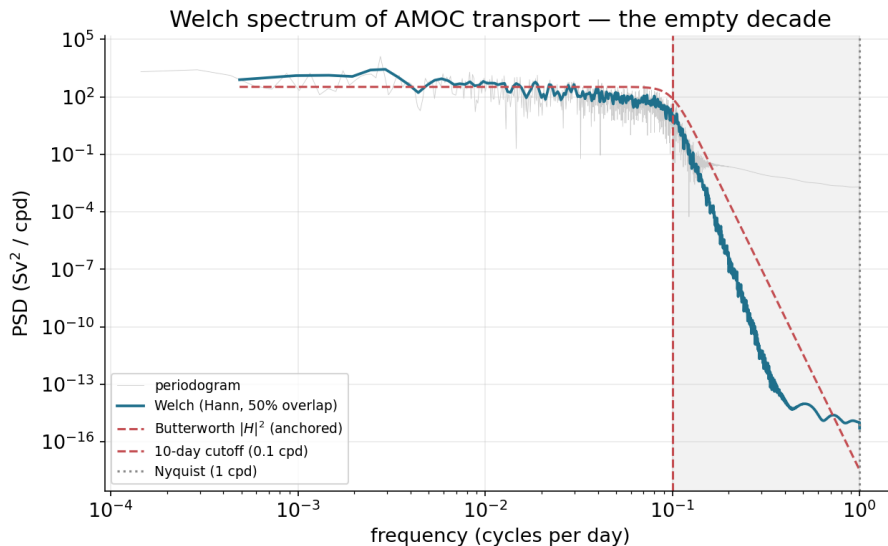
- Split the record into overlapping (50%) tapered segments
- Average their periodograms
- Trade frequency resolution for reduced variance and more d.o.f.

Welch, illustrated



split into overlapping tapered segments → average their periodograms

Welch spectrum: the empty decade



Flat to 0.1 cpd, then plunges — nothing between 0.1 and 1 cpd

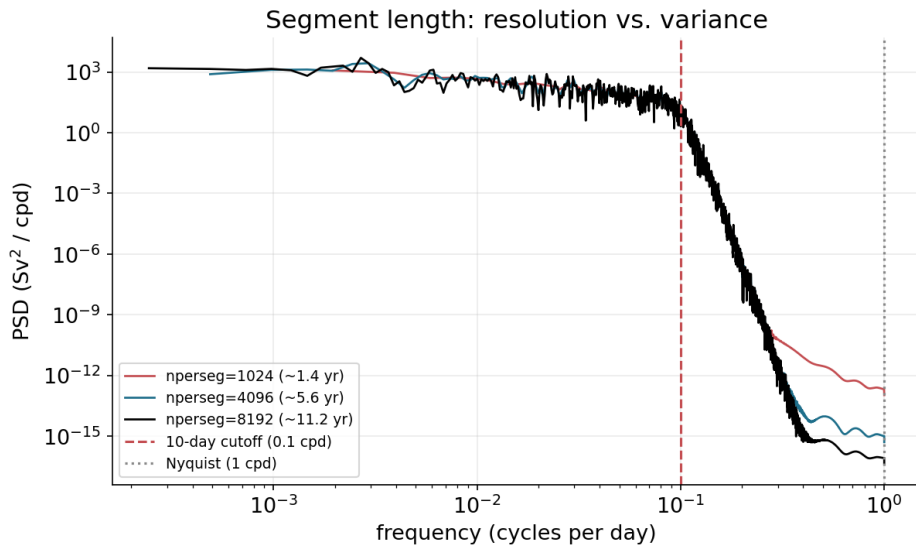
Caveat: the observed rolloff isn't the filter alone

- The Welch spectrum you just saw rolls off faster than the Butterworth $|H(f)|^2$ overlay
- AMOC transport is red (more variance at low frequencies), so the output rolloff = filter $\times$ red background
- The honest diagnostic is the half-power point (≈ 0.1 cpd), not the full slope
- So don't read the filter's response directly off the measured spectrum

Code — Welch spectrum

```
f, Pxx = signal.welch(  
    x, fs=fs, window="hann",  
    nperseg=4096, noverlap=2048,  
    detrend="linear")  
  
from scipy.stats import chi2      # confidence band  
K = 5; dof = 2 * K               # ~segments -> d.o.f.  
lo = dof / chi2.ppf(0.975, dof)  
hi = dof / chi2.ppf(0.025, dof)  
plt.fill_between(f, Pxx*lo, Pxx*hi, alpha=0.2)
```

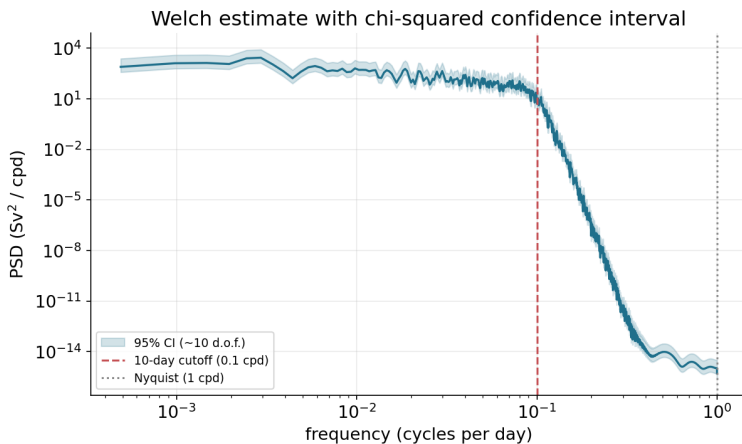
Choosing the segment length



Longer segments: finer resolution but fewer averages

Confidence intervals (chi-squared)

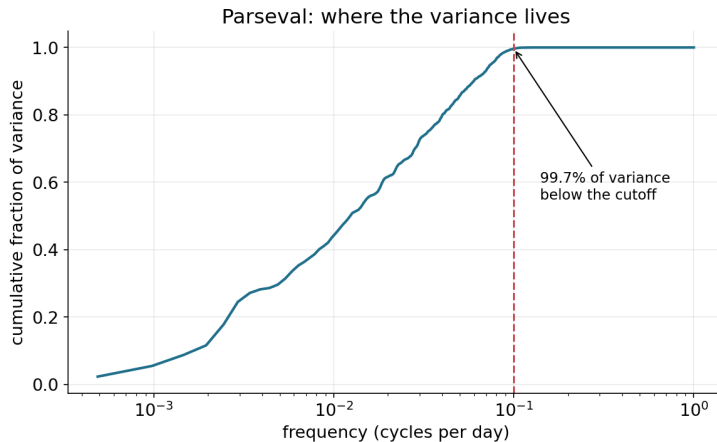
$$S_{\text{lo}} = \frac{\nu \hat{S}}{\chi_{\nu, 0.975}^2}, \quad S_{\text{hi}} = \frac{\nu \hat{S}}{\chi_{\nu, 0.025}^2}, \quad \nu = 2K$$



$\nu = 2K$ degrees of freedom (K = number of averaged segments)

Parseval: where the variance lives

$$\text{var}(x) = \int_0^{f_N} S(f) df \approx \sum_k S(f_k) \Delta f$$



About 99.7% of the variance sits below the 0.1 cpd cutoff

Code — Parseval check

```
df = f[1] - f[0]
var_psd = np.sum(Pxx) * df
print(var_psd, np.var(x))          # approx equal

frac = np.cumsum(Pxx) * df / var_psd
```

Rank filters on spectral grounds

4 · DESIGNING FILTERS THAT DON'T LIE

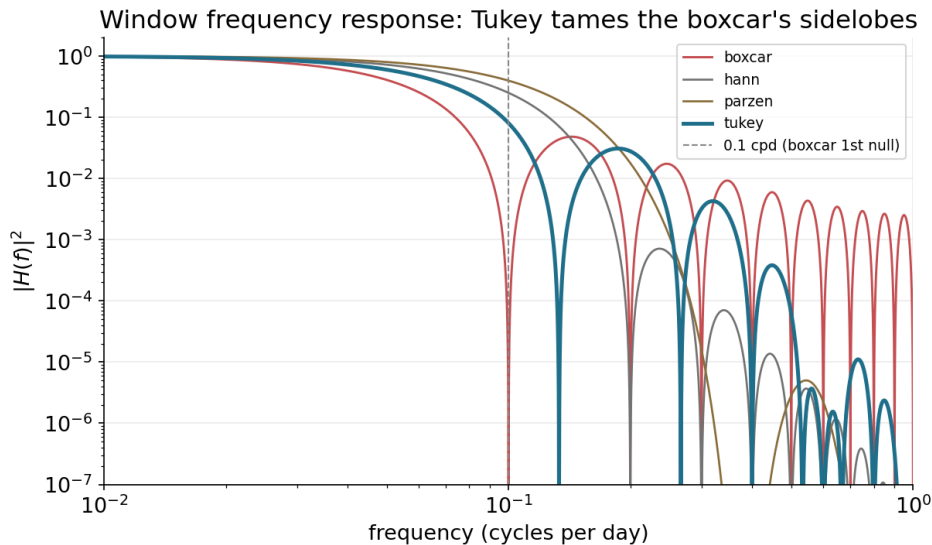
Two faces of tapering

- Taper the data segment before the FFT $\rightarrow$ less leakage in estimation
- Taper the filter weights in design $\rightarrow$ less stopband leakage in filtering
- Same maths, two applications students routinely conflate

The filter family

- Filter by convolving a window — a running, weighted mean
- Boxcar = plain running mean: leaky sinc sidelobes (cautionary case)
- Taper the window (Hann, Parzen, Tukey) to suppress the sidelobes
- Tukey: boxcar-like but less jaggedy, with a clean frequency response

Window frequency response: sidelobes are the diagnostic

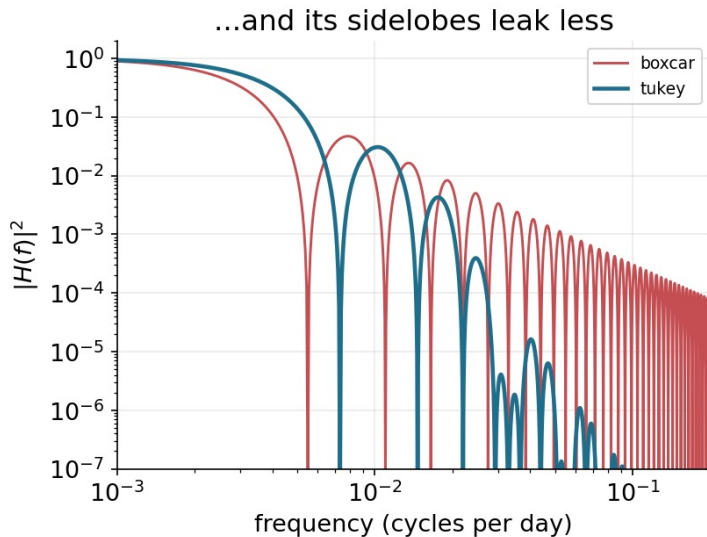
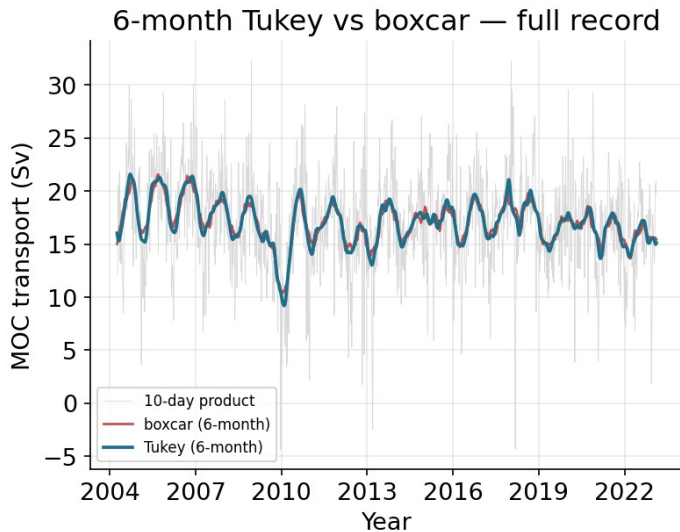


Boxcar leaks; Hann, Parzen and Tukey roll off cleanly

Code — window responses

```
for win in ("boxcar", "hann", "parzen", "tukey"):
    w = signal.get_window(win, 20)    # 10-day
    w = w / w.sum()
    fr, H = signal.freqz(w, worN=8192, fs=fs)
    plt.loglog(fr, np.abs(H)**2, label=win)
```

Tukey rolling mean: smoother, and it leaks less



6-month Tukey vs boxcar over the full record — `pandas rolling(win_type='tukey')`

Code — Tukey filter (pandas)

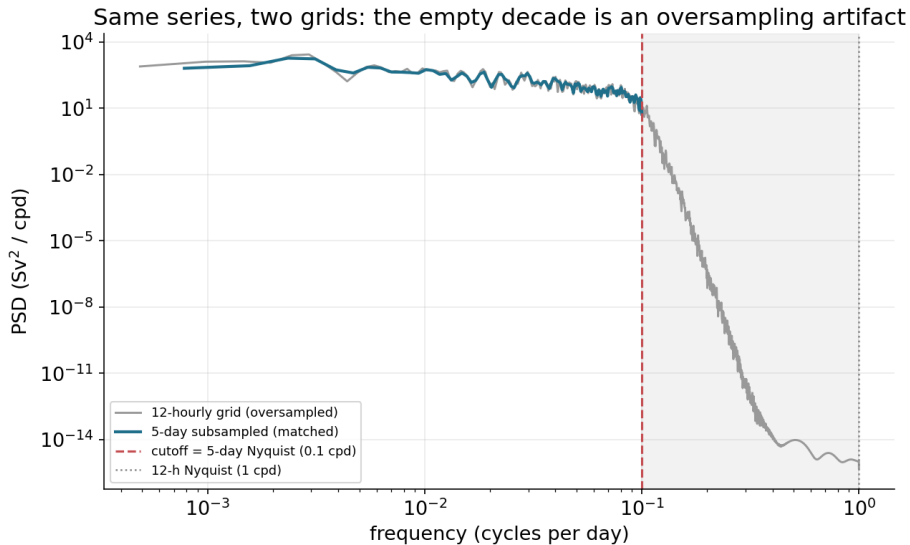
```
import pandas as pd
s = pd.Series(x)

box = s.rolling(40, center=True,
               win_type="boxcar", min_periods=1).mean()
tuk = s.rolling(40, center=True,
               win_type="tukey", min_periods=1).mean()
```

The grid is fine; the information is coarse

5 · OVERSAMPLING MADE VISIBLE

Same series, two grids

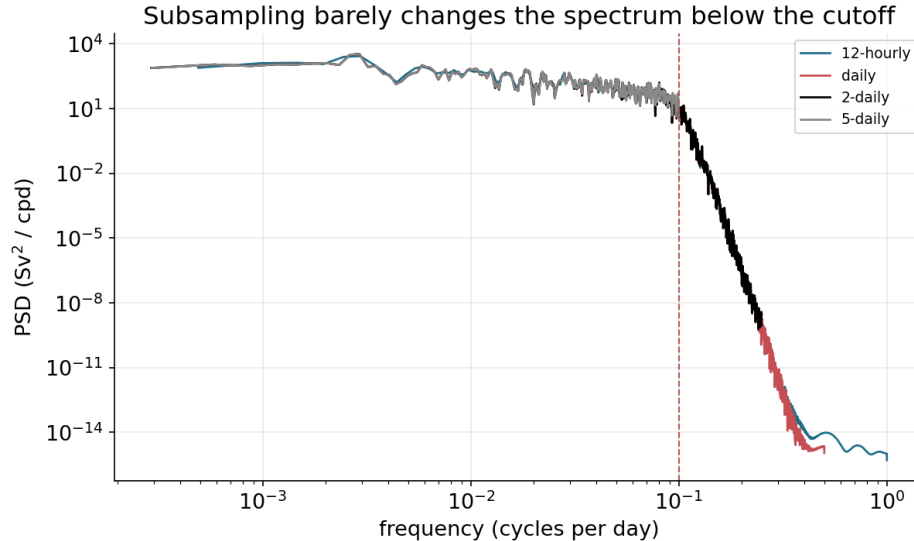


The empty decade is an artifact of the 12-hour grid

Code — subsampling contrast

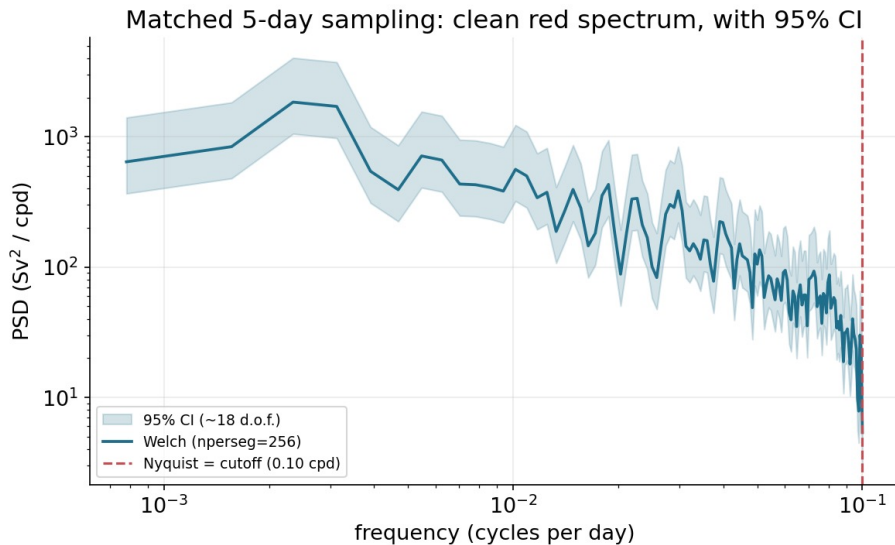
```
for step, lab in [(1,"12-hourly"), (10,"5-day")]:  
    xs = x[:,::step]  
    fss = fs / step  
    fr, P = signal.welch(  
        xs, fs=fss, window="hann",  
        nperseg=min(4096, len(xs)//4*2))  
    plt.loglog(fr, P, label=lab)
```

Subsampling barely moves the low-f spectrum



Daily, 2-daily and 5-daily all agree below the cutoff

Matched 5-day sampling: a normal red spectrum



Matched 5-day sampling, now with a 95% chi-squared confidence interval

Effective resolution ≈ 10 days

- The 12-hour grid carries no independent high-frequency information
- Effective decorrelation timescale ≈ 10 days
- The grid is fine; the information is coarse

Where next: spectral coherence

- Once you can estimate a spectrum, ask whether two series covary at a frequency
- Coherence = correlation as a function of frequency
- e.g. Ekman (t_{ek10}) vs total transport — cross-spectral methods, next lecture

Recap: the spine

- Grid: what could be resolved (Nyquist)
- Filter: what is actually there (10-day Butterworth)
- Honest estimate: periodogram $\rightarrow$ taper $\rightarrow$ Welch $\rightarrow$ CIs $\rightarrow$ Parseval
- Honest filter design: Butterworth / Lanczos / Loess vs boxcar
- Payoff: the empty decade between 0.1 and 1 cpd

References

- Emery & Thomson, Data Analysis Methods in Physical Oceanography, Ch. 5 (§ 5.6 Spectral analysis; § 5.10 Digital filters)
- J. M. Lilly, course notes on spectra & autocovariance (jmlilly.net)
- McCarthy et al. 2015, Measuring the AMOC at 26° N (Prog. Oceanogr.)
- RAPID 26° N transport data, doi:10.5285/223b34a32dc5c945e0637086abc0f274

We're looking for glider pilots

- Interested in piloting ocean gliders? Come talk to Chiara.
- Curious what it's actually like day-to-day? Ask Isabelle.